

# CS 3793

# Artificial Intelligence

# Fall 2022

Predictive Analytics :Understanding  
Customer Behavior  
Through Machine Learning  
(Unsupervised Learning Algorithms)

## Group 7 Members:

Alexander  
Cepeda

Maria Jose  
Healey

Jongbo Im

Meera Patel

Lyann Serrano

# Introduction - The problem

- ▶ A 2020 study found that the US is home to over 1 million business establishments.
- ▶ This competitive drive between these establishments has led to the use of **Predictive Analytics**, defined as the use of statistics and modeling techniques to determine future performance based on current and historical data.
- ▶ This tech helps maximizes profit, optimize resources, and nail marketing strategies.
- ▶ Our goal is to introduce you to some of these predictive analytics strategies that aim to sky rocket a company to its full potential.

# Introduction - Why it matters.

## The business aspect

- ▶ Businesses without knowledge of predictive analytics would miss out on countless opportunities to reach their full potential. This may include missed opportunities to...
  1. Maximize customer value
  2. Decrease financial fraud
  3. Manage resources and inventory
- ▶ Not optimizing these business features could cause a company to die out to competitors.

# Predictive Analytics

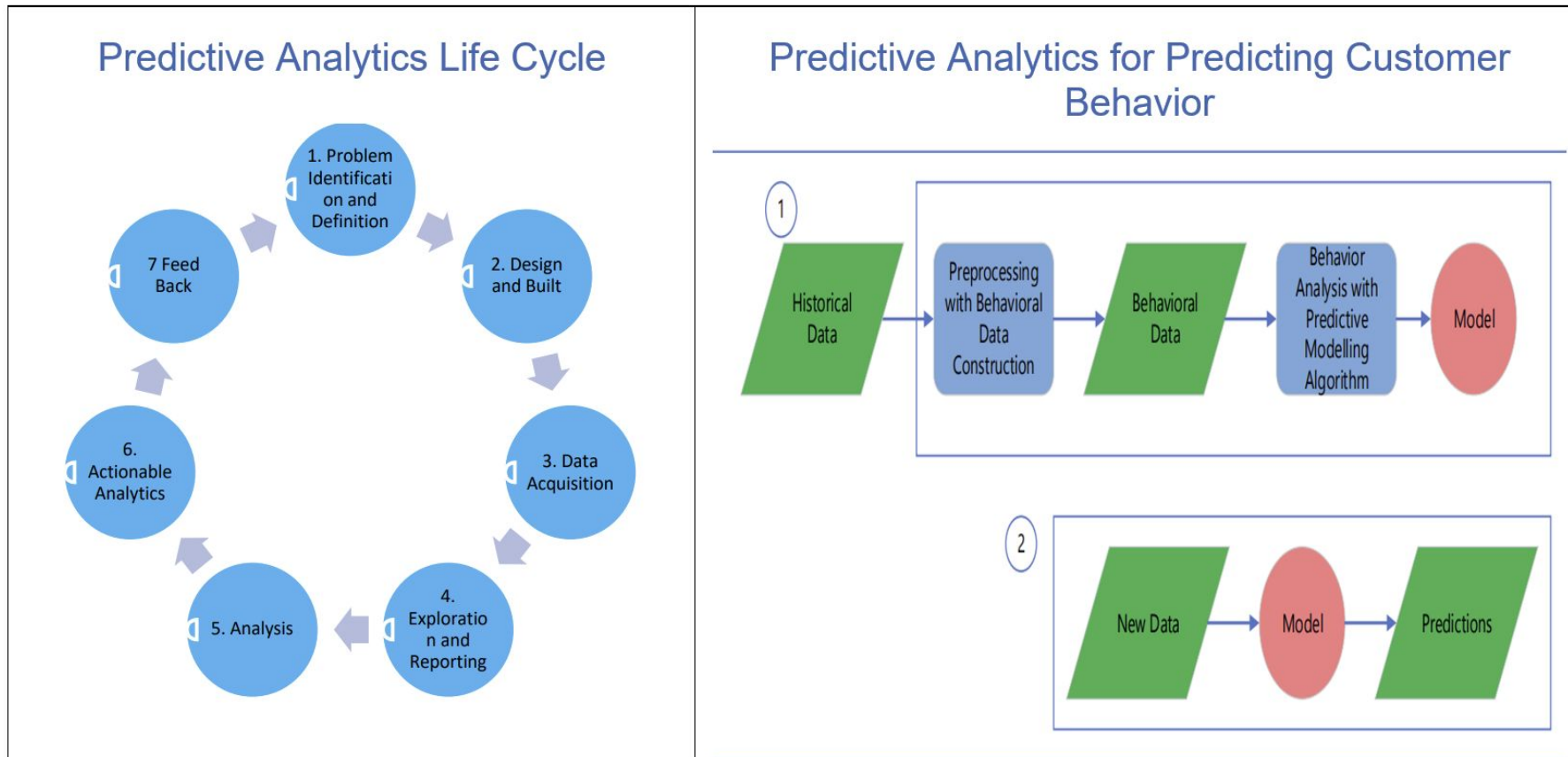


Fig 1. [1]

# Customer Segmentation

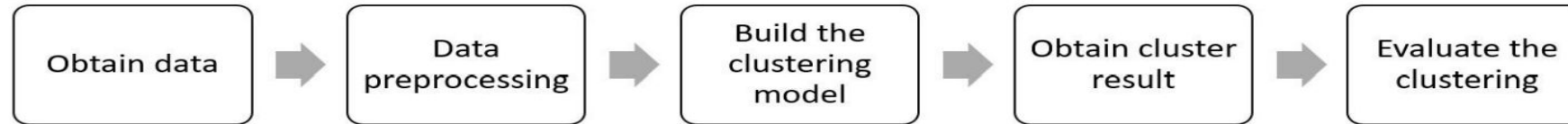


Fig 2. Applying K-Means algorithm to Customer Segmentation [2]

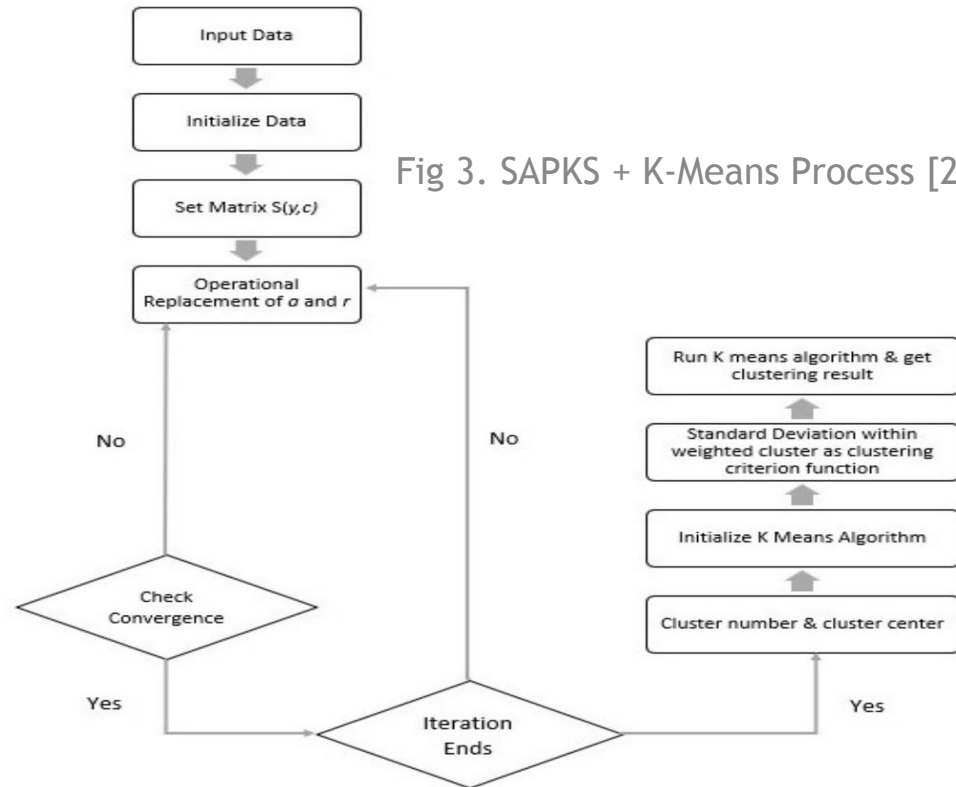


Fig 3. SAPKS + K-Means Process [2]

# Understanding Customer Behavior

## K-Means algorithm

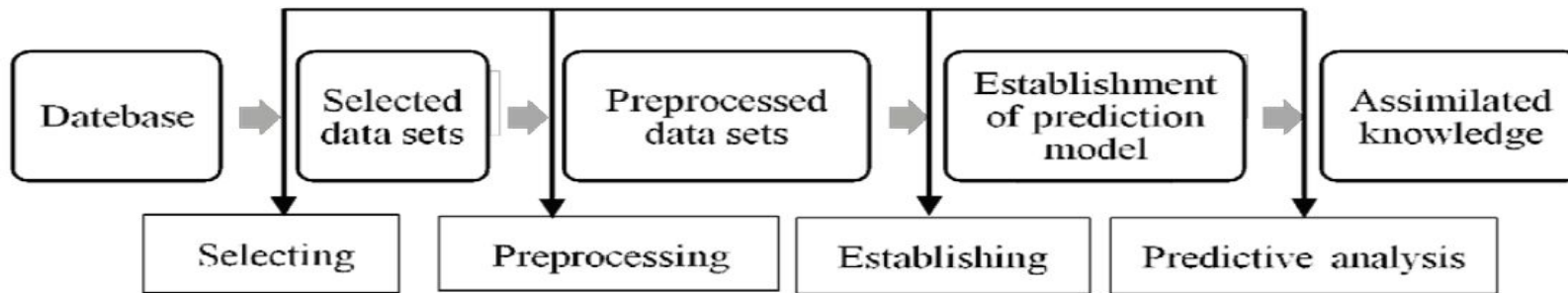
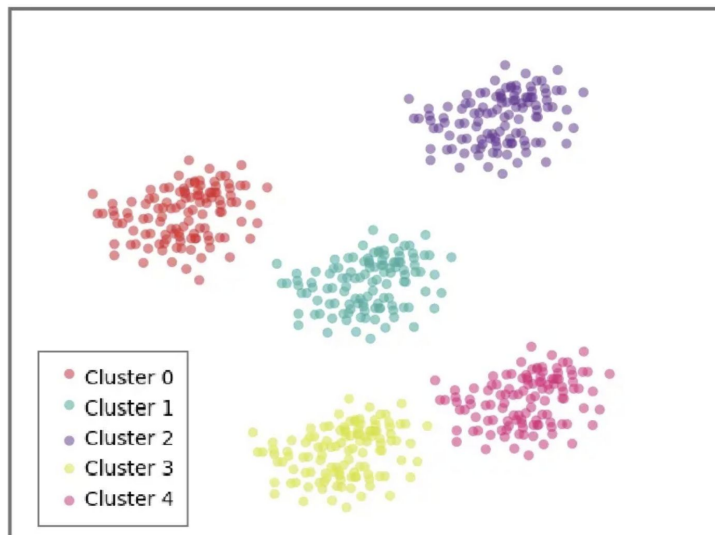


Fig 4. Prediction Process of Understanding Customer Behavior (e.g. Purchase) [3]



Healey

Fig 5. Clustering Example [4]

|   | CustomerID | Gender | Age | Annual Income (k\$) | Spending Score (1-100) |
|---|------------|--------|-----|---------------------|------------------------|
| 0 | 1          | Male   | 19  | 15                  | 39                     |
| 1 | 2          | Male   | 21  | 15                  | 81                     |
| 2 | 3          | Female | 20  | 16                  | 6                      |
| 3 | 4          | Female | 23  | 16                  | 77                     |
| 4 | 5          | Female | 31  | 17                  | 40                     |

Fig 6. Sample Data Set [4]

# Understanding Customer Behavior

## K-Means algorithm

Descriptive Statistics and Distribution [4]

```
# Descriptive statistics (mean, standard deviation, median and variance). If the variable is not numeric, gets the counts in each category.
def statistics(variable):
    if variable.dtype == "int64" or variable.dtype == "float64":
        return pd.DataFrame([[variable.name, np.mean(variable), np.std(variable), np.median(variable), np.var(variable)]],
                             columns = ["Variable", "Mean", "Standard Deviation", "Median", "Variance"]).set_index("Variable")
    else:
        return pd.DataFrame(variable.value_counts())
```

```
spending = customers["Spending Score (1-100)"]
```

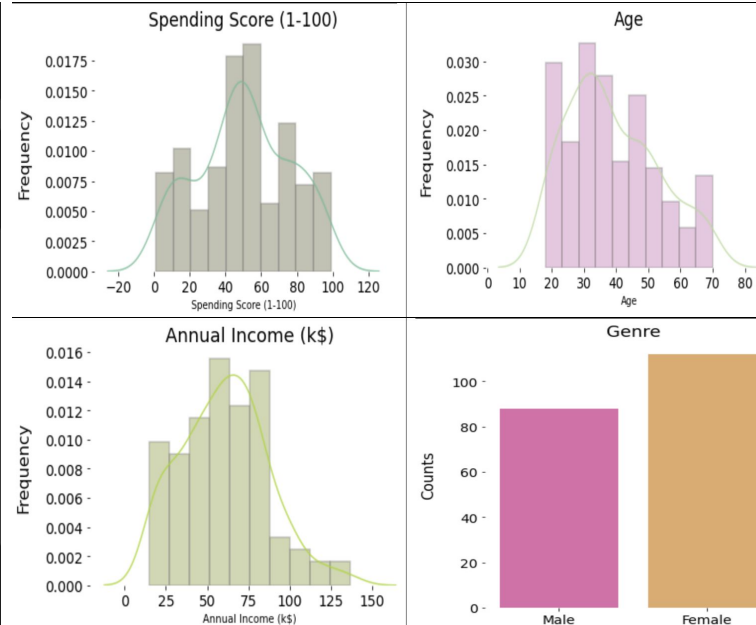
```
statistics(spending)
```

|  | Mean | Standard Deviation | Median | Variance |
|--|------|--------------------|--------|----------|
|--|------|--------------------|--------|----------|

| Variable | Mean | Standard Deviation | Median | Variance |
|----------|------|--------------------|--------|----------|
|----------|------|--------------------|--------|----------|

|                        |      |           |      |        |
|------------------------|------|-----------|------|--------|
| Spending Score (1-100) | 50.2 | 25.758882 | 50.0 | 663.52 |
|------------------------|------|-----------|------|--------|

```
graph_histo(spending)
```

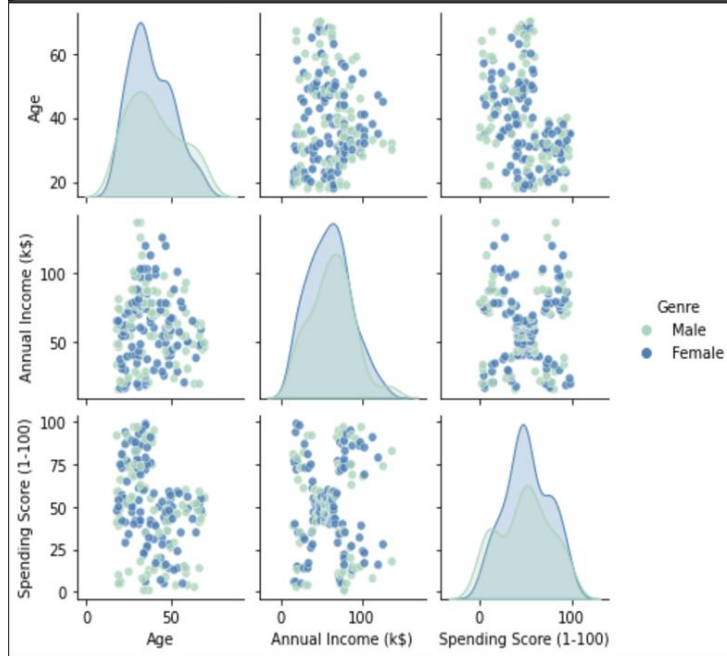


# Understanding Customer Behavior

## K-Means algorithm

Correlation between Parameters and Dimensionality Reduction [4]

```
# Analyzes the correlation between the numeric parameters. For that aim, uses the pairplot seaborn function.
# Wants to see whether there is a difference between gender. So, the hue parameter will be used to get different colors for points belonging to female or male cust
sns.pairplot(customers, x_vars = ["Age", "Annual Income (k$)", "Spending Score (1-100)"],
             y_vars = ["Age", "Annual Income (k$)", "Spending Score (1-100)"],
             hue = "Genre",
             kind = "scatter",
             palette = "YlGnBu",
             height = 2,
             plot_kws={"s": 35, "alpha": 0.8});
```



```
# Principal Component Analysis (PCA) - Transforms the categorical variable into two binary variables.
```

```
customers["Male"] = customers.Genre.apply(lambda x: 0 if x == "Male" else 1)
customers["Female"] = customers.Genre.apply(lambda x: 0 if x == "Female" else 1)
```

```
# Selects from the dataset all the useful columns. Customer ID is not a useful feature. Gender will split it into two binaries categories
```

```
X = customers.iloc[:, 2:]
```

```
X.head()
```

|   | Age | Annual Income (k\$) | Spending Score (1-100) | Male | Female |
|---|-----|---------------------|------------------------|------|--------|
| 0 | 19  | 15                  | 39                     | 0    | 1      |
| 1 | 21  | 15                  | 81                     | 0    | 1      |
| 2 | 20  | 16                  | 6                      | 1    | 0      |
| 3 | 23  | 16                  | 77                     | 1    | 0      |
| 4 | 31  | 17                  | 40                     | 1    | 0      |

```
# Uses the PCA function from sklearn module - Applies PCA and fits the features selected
```

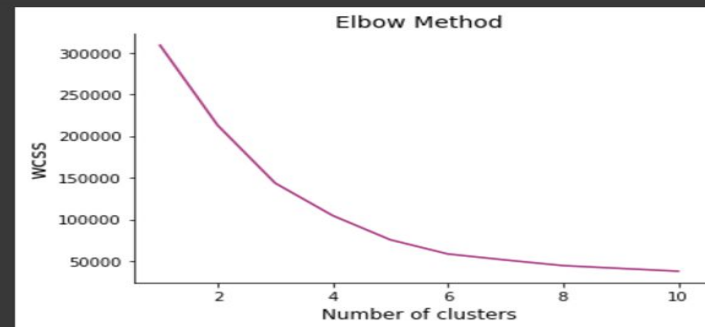
```
pca = PCA(n_components=2).fit(X)
```

```
# Transforms samples using the PCA fit
pca_2d = pca.transform(X)
```

# Understanding Customer Behavior

## K-Means Clustering [4]

```
wcss = []
for i in range(1,11):
    km = KMeans(n_clusters=i,init='k-means++', max_iter=300, n_init=10, random_state=0)
    km.fit(X)
    wcss.append(km.inertia_)
plt.plot(range(1,11),wcss, c="#c51b7d")
plt.gca().spines["top"].set_visible(False)
plt.gca().spines["right"].set_visible(False)
plt.title('Elbow Method', size=14)
plt.xlabel('Number of clusters', size=12)
plt.ylabel('wcss', size=14)
plt.show()
```

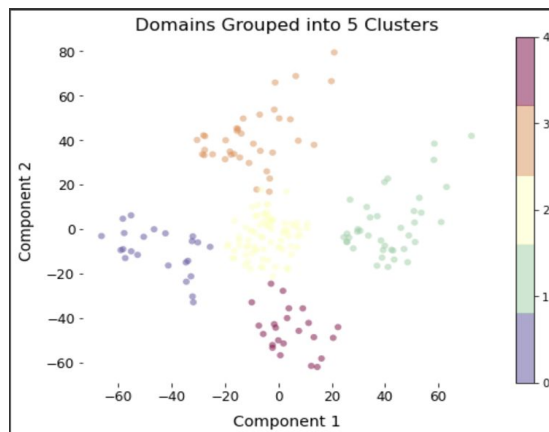


```
# Kmeans algorithm
# n_clusters: Number of clusters. In our case 5
# init: k-means++. Smart initialization
# max_iter: Maximum number of iterations of the k-means algorithm for a single run
# n_init: Number of time the k-means algorithm will be run with different centroid seeds.
# random_state: Determines random number generation for centroid initialization.
kmeans = KMeans(n_clusters=5, init='k-means++', max_iter=10, n_init=10, random_state=0)

# Fit and predict
y_means = kmeans.fit_predict(X)
```

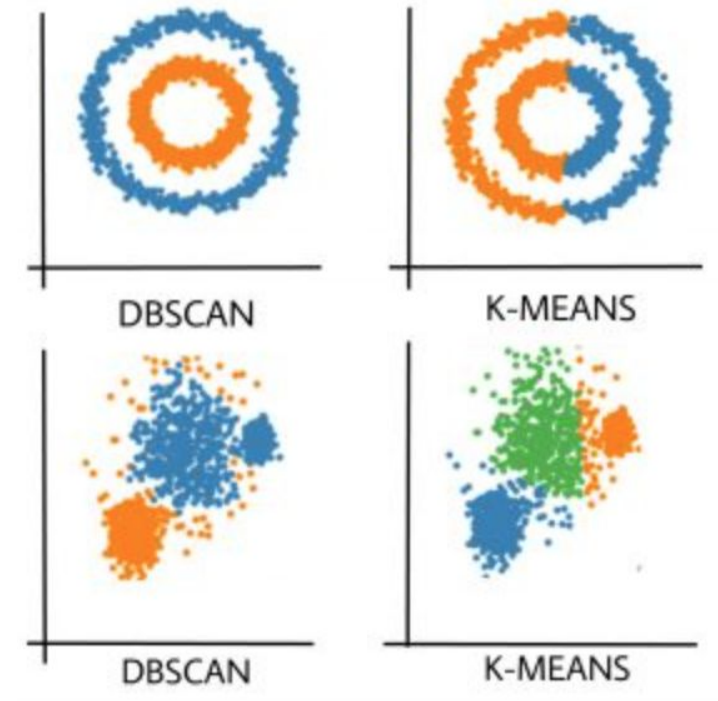
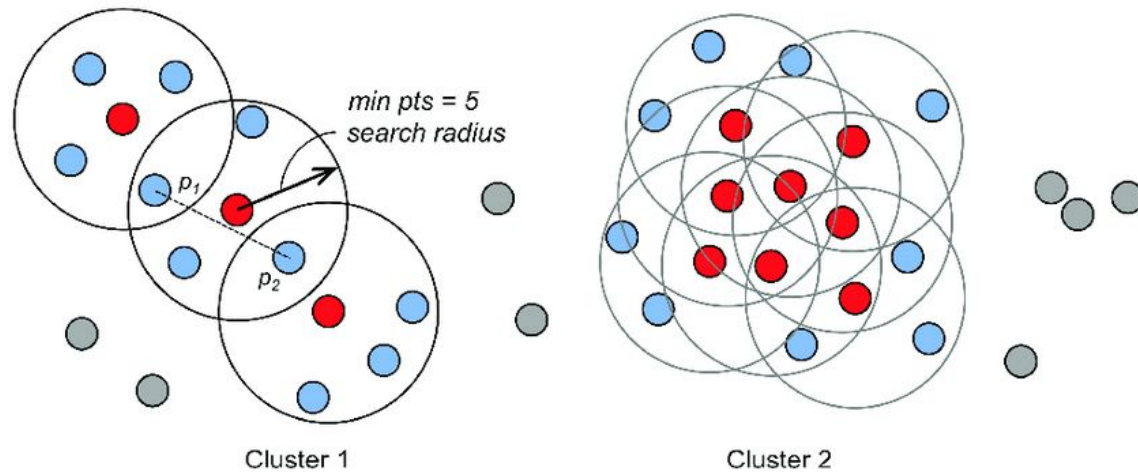
```
centroids = pd.DataFrame(kmeans.cluster_centers_, columns = ["Age", "Annual Income", "Spending", "Male", "Female"])
centroids.index_name = "ClusterID"
centroids["ClusterID"] = centroids.index
centroids = centroids.reset_index(drop=True)
centroids
```

|   | Age       | Annual Income | Spending  | Male     | Female   | ClusterID |
|---|-----------|---------------|-----------|----------|----------|-----------|
| 0 | 45.217391 | 26.304348     | 20.913043 | 0.608696 | 0.391304 | 0         |
| 1 | 32.692308 | 86.538462     | 82.128205 | 0.538462 | 0.461538 | 1         |
| 2 | 43.088608 | 55.291139     | 49.569620 | 0.582278 | 0.417722 | 2         |
| 3 | 40.666667 | 87.750000     | 17.583333 | 0.472222 | 0.527778 | 3         |
| 4 | 25.521739 | 26.304348     | 78.565217 | 0.608696 | 0.391304 | 4         |



# Understanding Customer Behavior

## Density Based Clustering

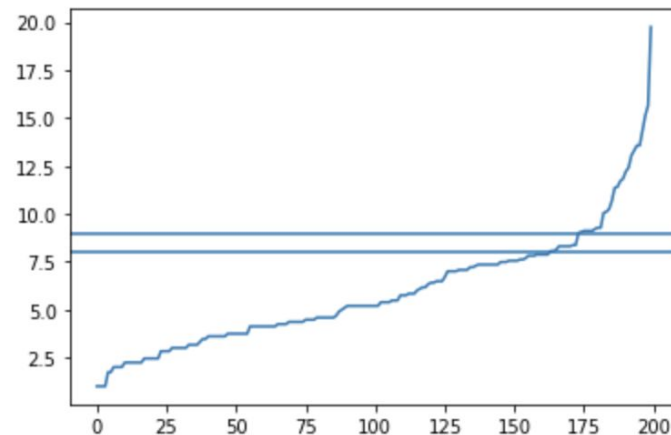


# Understanding Customer Behavior

## DBSCAN

```
1 from sklearn.neighbors import NearestNeighbors
2
3 neighbors = NearestNeighbors(n_neighbors=20)
4 neighbors_fit = neighbors.fit(X)
5 distances, indices = neighbors_fit.kneighbors(X)
6
7 distances = np.sort(distances, axis=0)
8 distances = distances[:,1]
9 plt.plot(distances)
10
11 plt.axhline(9)
12 plt.axhline(8)
```

<matplotlib.lines.Line2D at 0x7fd5779c70>



Reference for code below

<https://medium.com/@tarammullin/dbscan-parameter-estimation-ff8330e3a3bd>

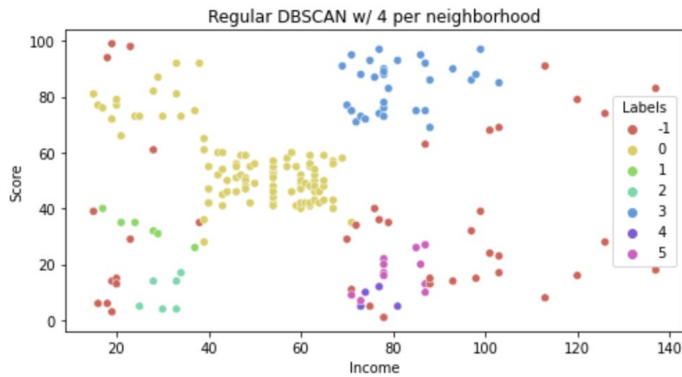
# Understanding Customer Behavior

## DBSCAN

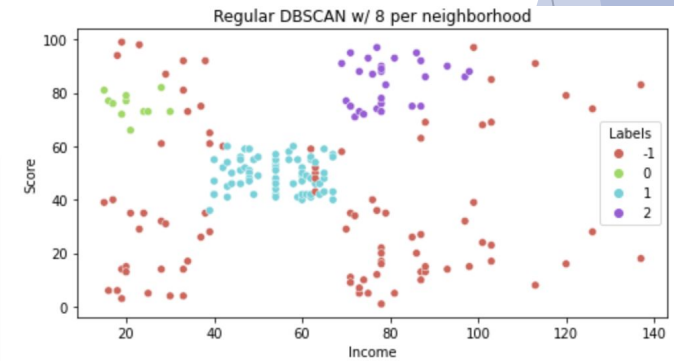
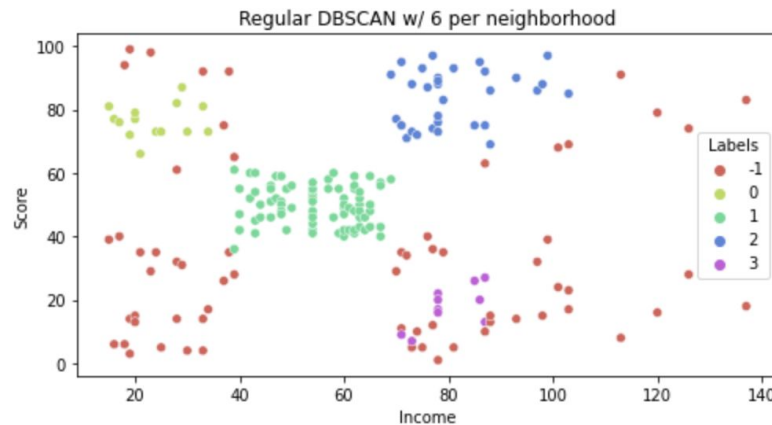
```
1 X = df.drop(['CustomerID', 'Genre'], axis=1)
2
3 db = DBSCAN(eps=11, min_samples=6).fit(X)
4
5 X['Labels'] = db.labels_
6 plt.figure(figsize=(8, 4))
7 sns.scatterplot(X['Income'], X['Score'], hue=X['Labels'], palette=sns.color_palette('hls', np.unique(db.labels_).shape[0]))
8 plt.title('Regular DBSCAN')
9 plt.show()
```

This section is going to be used to compare the improved dbscan

<https://www.kaggle.com/code/fazilbtopal/popular-unsupervised-clustering-algorithms/notebook>



Serrano



13

# Implementation of Improved-DBSCAN

A skeleton of DBSCAN is used as a base to start the implementation of improved DBSCAN which improves the efficiency of clustering.

<https://becominghuman.ai/dbscan-clustering-algorithm-implementation-from-scratch-python-9950af5eed97>

```
1 def check_core_point(eps,minPts, df, seedsnum): #seedsnum is the number of points used as seeds
2     #get points from given index
3     x, y = df.iloc[seedsnum]['X'] , df.iloc[seedsnum]['Y']
4
5     #check available points within radius
6     temp = df[((np.abs(x - df['X']) <= eps) & (np.abs(y - df['Y']) <= eps)) & (df.index != seedsnum)]
7
8     #check how many points are present within radius
9     if len(temp) >= minPts:
10         #return format (dataframe, is_core, is_border, is_noise)
11         return (temp.index , True, False, False)
12
13     elif (len(temp) < minPts) and len(temp) > 0:
14         #return format (dataframe, is_core, is_border, is_noise)
15         return (temp.index , False, True, False)
16
17     elif len(temp) == 0:
18         #return format (dataframe, is_core, is_border, is_noise)
19         return (temp.index , False, False, True) #output category labels
20
21
22 def cluster_with_stack(eps, minPts, df, seedsnum):
23
24     current_stack = set()
25     unvisited = list(df.index)
26     clusters = []
27     labels=np.empty(len(unvisited));
28     labels.fill(-2)
29
30     while (len(unvisited) != 0): #run until all points have been visited
31         seeds = #select seedsnum data points from UD as seeds
32
33         #identifier for first point of a cluster
34         first_point = True
35
36         #choose a random unvisited point
37         current_stack.add(random.choice(unvisited))
```

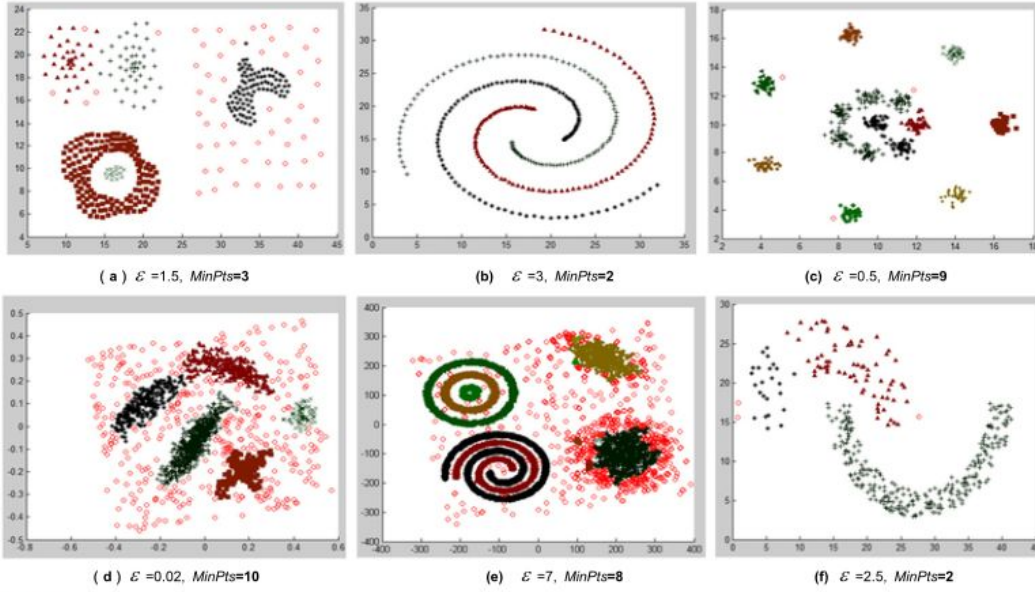


FIGURE 1. The clustering results of the proposed algorithm on some experiments on toy data sets. Red hollow circles are noise identified by the proposed algorithm.

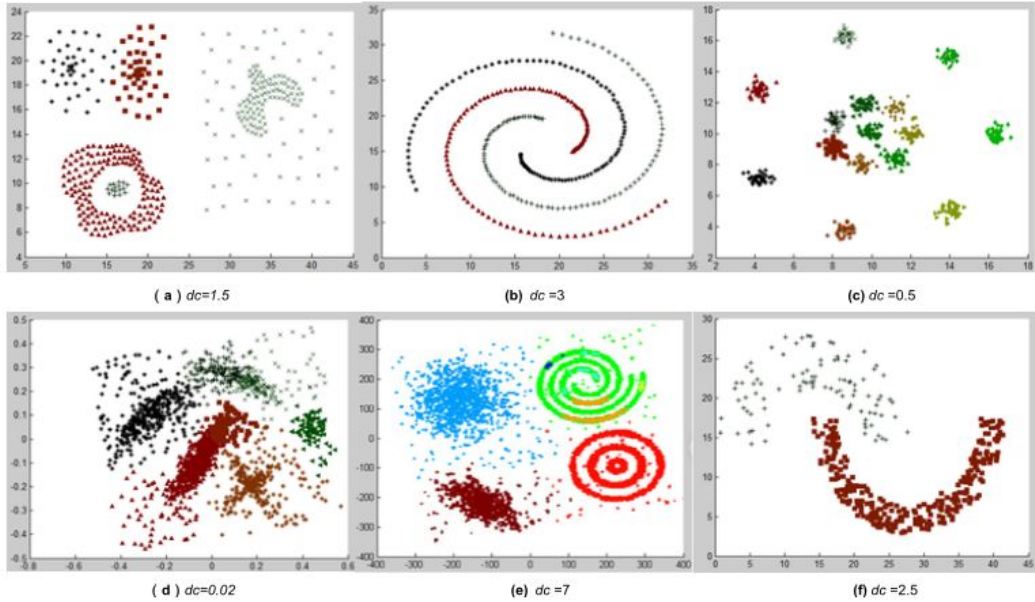


FIGURE 2. The clustering results of DPeak on some experiments on toy data sets, where  $dc$  is the scanning radius,  $\epsilon$ , of DPeak.

#### Algorithm 1 ImprovedDBSCAN

**Input:** data set  $D$ , radius  $\epsilon$ , density threshold  $MinPts$ ,  $seedsnum$  //  $seedsnum$  is the number of points that are used as seeds

**Output:** category labels  $Labels$

1 Build a hierarchical covering tree  $Tl$  for dataset  $D$ ;

2 Initialization:

2.1 Set unprocessed data set  $UD = D$

2.2 Set the neighbor set of each point as  $TempN_\epsilon(p) := \{\}$

2.3  $Labels = \{-2, -2, \dots, -2\}$

3 While  $UD$  is not NULL

3.1  $Seeds =$  select  $seedsnum$  data points from  $UD$  as seeds ;

3.2 Build query tree  $TQ$  for  $Seeds$

3.3 Use CoverTree:: RangeQuery ( $Tl, TQ, 2\epsilon$ ) to parallel retrieve  $2\epsilon$ -neighbors for each seed point in  $Seeds$

3.4 For each data point  $p$  in seeds

3.4.1 If  $|N_{2\epsilon}(p)| < MinPts$ ,

(1)  $Label(N_\epsilon(p)) = -1$  //Set the label of each point in  $N_\epsilon(p)$  to be “-1” which means it is an outlier (Theorem 3)

(2)  $UD = UD - N_\epsilon(p)$

End If

3.4.2 If  $|N_\epsilon(p)| \geq MinPts$

(1) Mark  $p$  as a core point,  $UD = UD - \{p\}$

(2)  $\forall s \in N_\epsilon(p)$ , find all  $\epsilon$ -nearest neighbors of  $s$  within  $N_\epsilon(p)$  according to Property 1-3, and add them into  $TempN_\epsilon(s)$

(3) If  $|TempN_\epsilon(s)| > MinPts$ , then

Mark  $s$  as a core point

$UD = UD - \{s\}$

End If

End If

End For

End While

4. While there is a core  $p$  unlabeled //The label of core point  $p$  is less than 0

4.1 current  $TempLabel = \text{Max}(Label) + 1$

4.2  $aCluster = \{\text{all core points that are density-reachable from } p\} \cup \{p\}$

4.3  $Label(aCluster) = TempLabel$

End While

5. For each non-outlier and non-core point  $p$

5.1  $aCore =$  Find the nearest core point within  $N_\epsilon(p)$

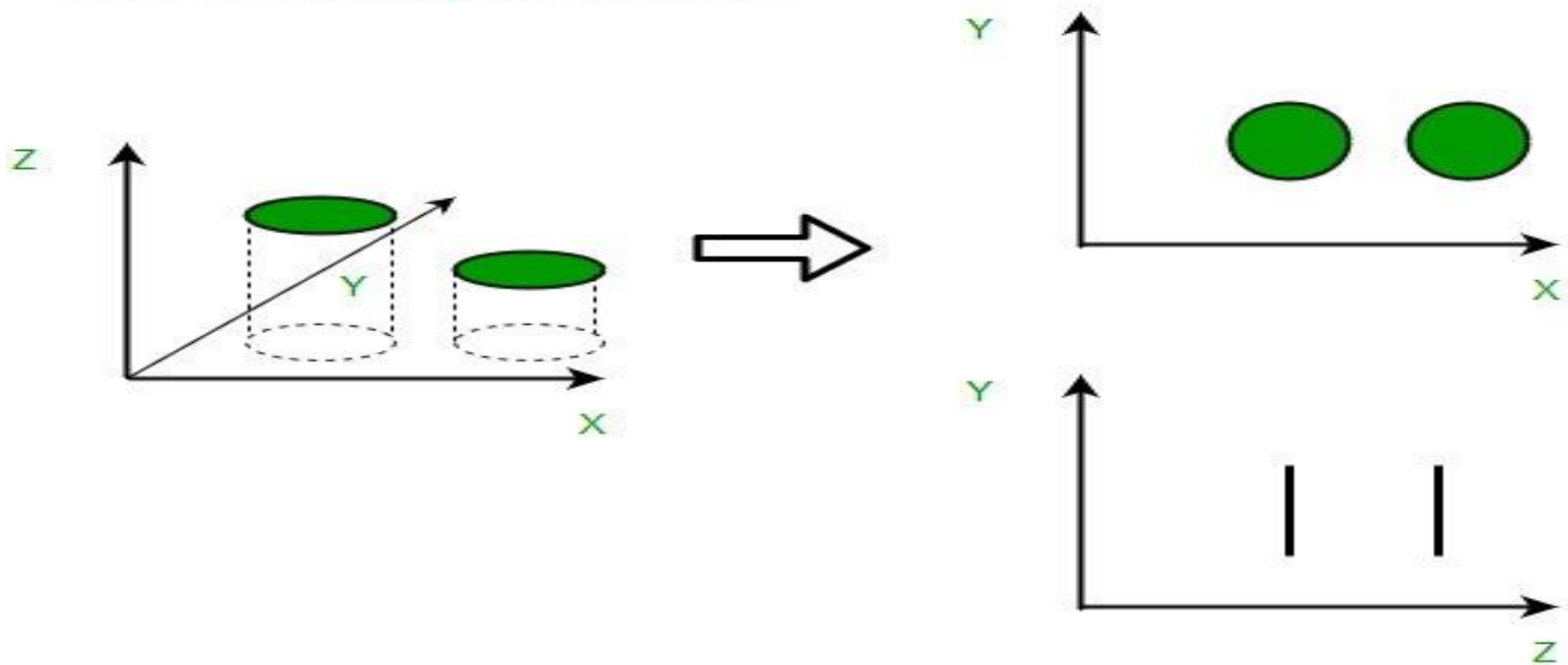
5.2  $Label(p) = Label(aCore)$  // assigned to the category of the core point

End For

End

# Locally Linear Embedding(LLE)

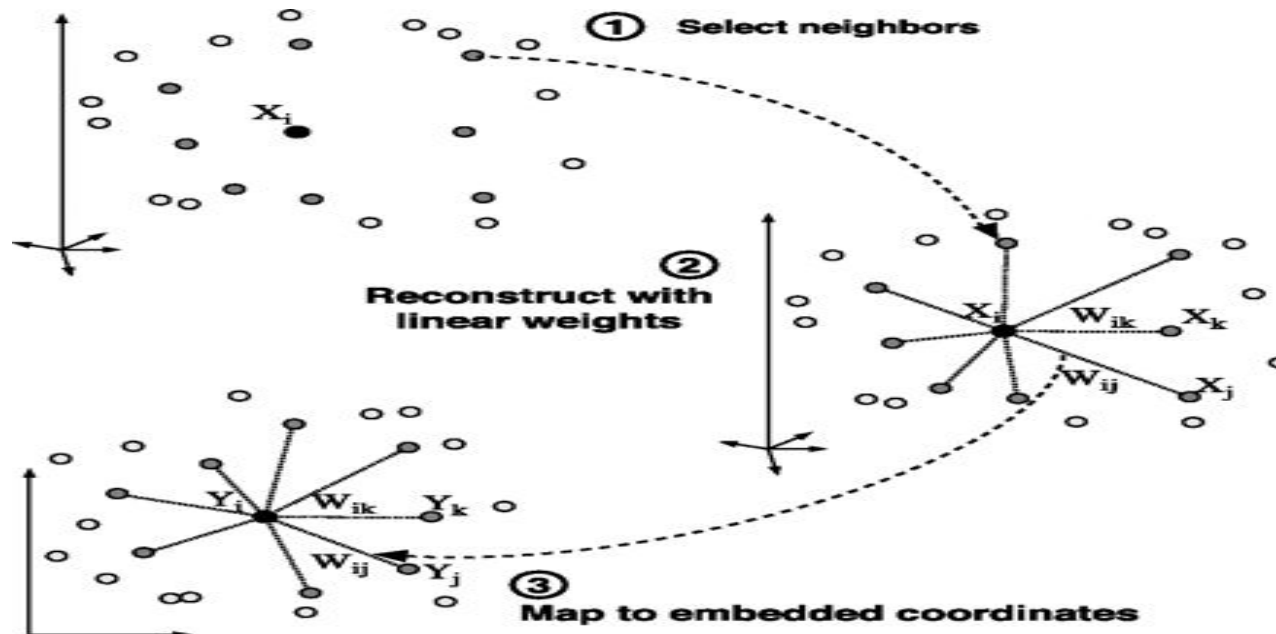
Dimensionality Reduction



# Locally Linear Embedding(LLE)

## How does it work?

- ▶ First LLE algorithm finds  $k$  points around point  $x$ , secondly it solves for the reconstruction weights lastly compute embedding coordinate  $Y$  using weight that is computed in step 2.



```
params = {
    "n_neighbors": n_neighbors,
    "n_components": n_components,
    "eigen_solver": "auto",
    "random_state": rng,
}

lle_standard = manifold.LocallyLinearEmbedding(method="standard", **params)
S_standard = lle_standard.fit_transform(S_points)

lle_lttsa = manifold.LocallyLinearEmbedding(method="lttsa", **params)
S_lttsa = lle_lttsa.fit_transform(S_points)

lle_hessian = manifold.LocallyLinearEmbedding(method="hessian", **params)
S_hessian = lle_hessian.fit_transform(S_points)

lle_mod = manifold.LocallyLinearEmbedding(method="modified", modified_tol=0.8, **params)
S_mod = lle_mod.fit_transform(S_points)
```

```
fig, axs = plt.subplots(
    nrows=2, ncols=2, figsize=(7, 7), facecolor="white", constrained_layout=True
)
fig.suptitle("Locally Linear Embeddings", size=16)

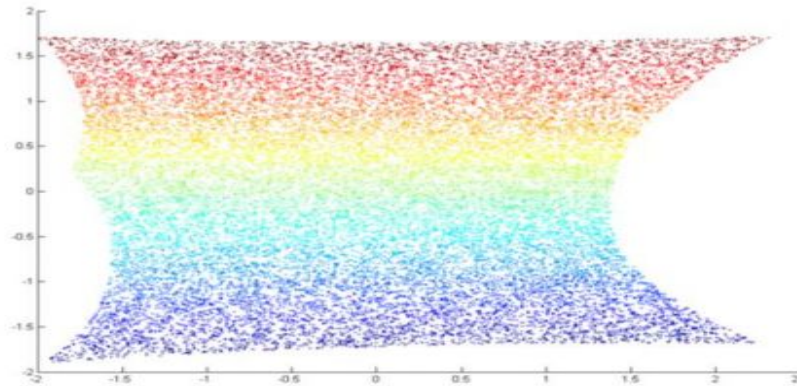
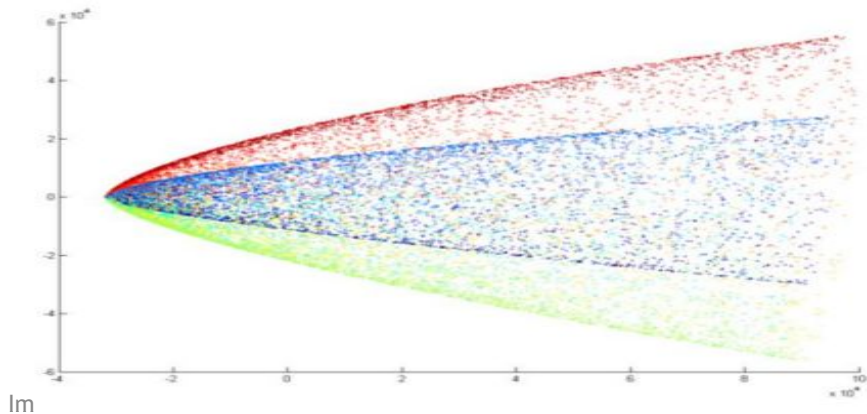
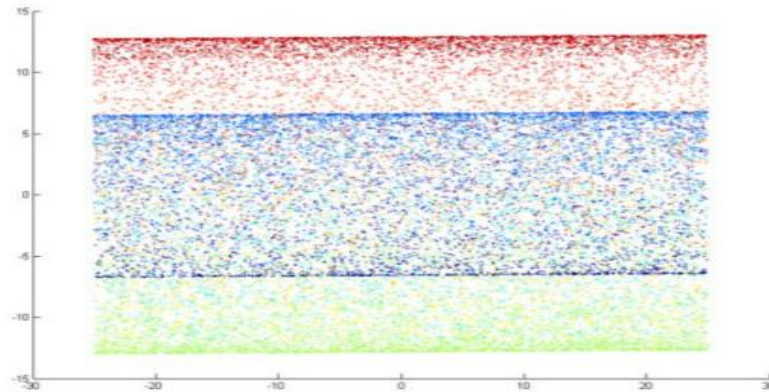
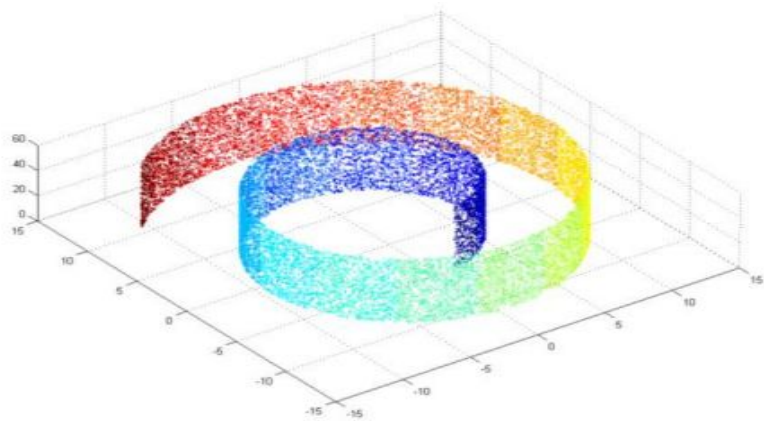
lle_methods = [
    ("Standard locally linear embedding", S_standard),
    ("Local tangent space alignment", S_lttsa),
    ("Hessian eigenmap", S_hessian),
    ("Modified locally linear embedding", S_mod),
]

for ax, method in zip(axs.flat, lle_methods):
    name, points = method
    add_2d_scatter(ax, points, S_color, name)

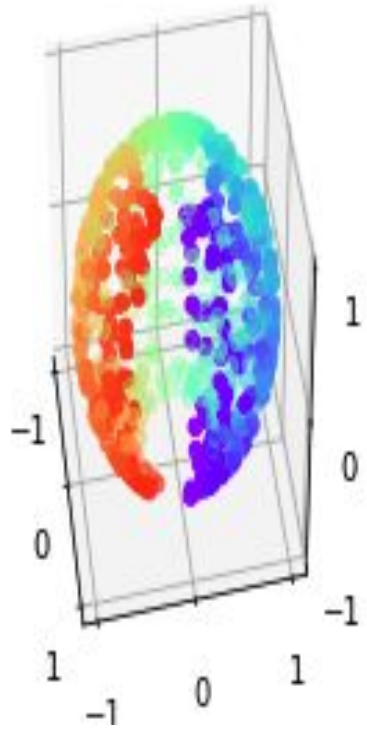
plt.show()
```

# Local Linear Embedding(LLE)

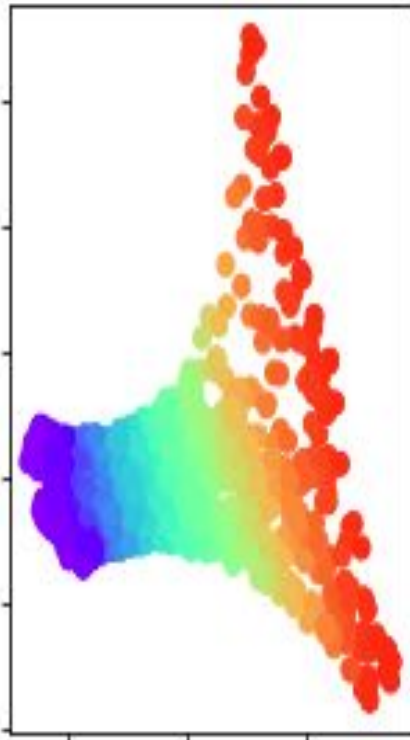
## Advantage of LLE algo. than other algo.



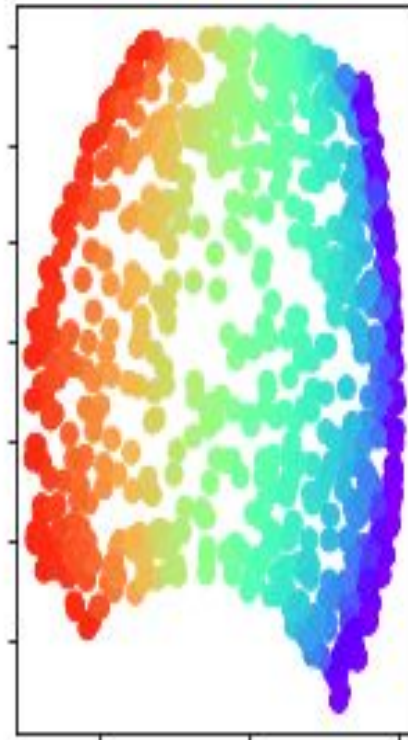
## Manifold Learning with 1000 points, 10 neighbors



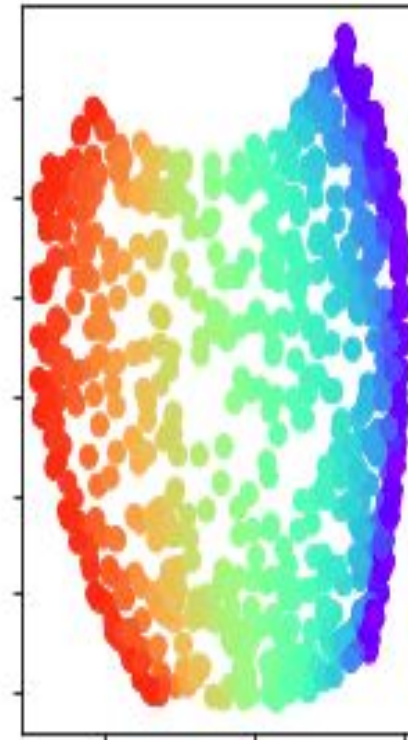
LLE (0.25 sec)



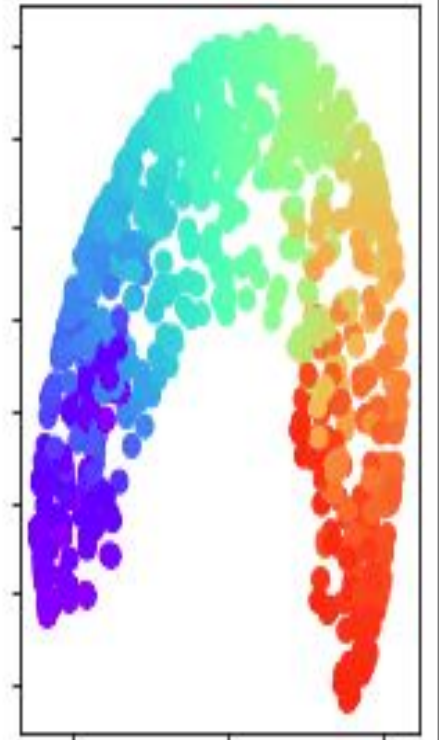
LTSA (0.16 sec)



Hessian LLE (0.17 sec)

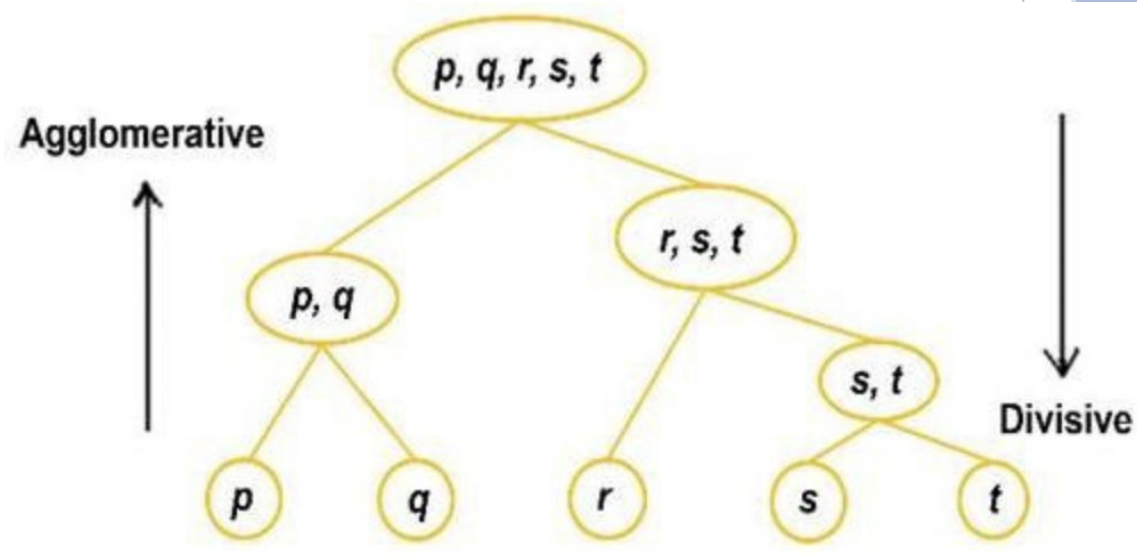
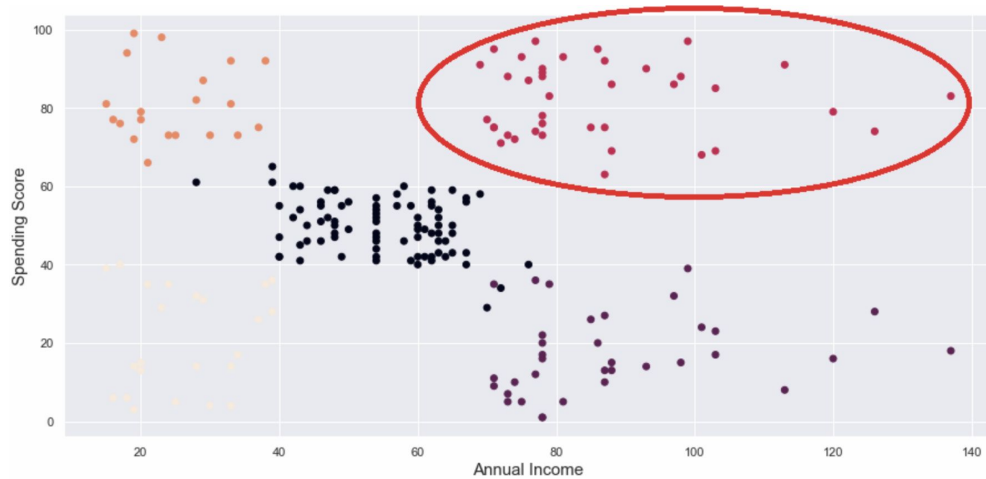


Modified LLE (0.14 sec)

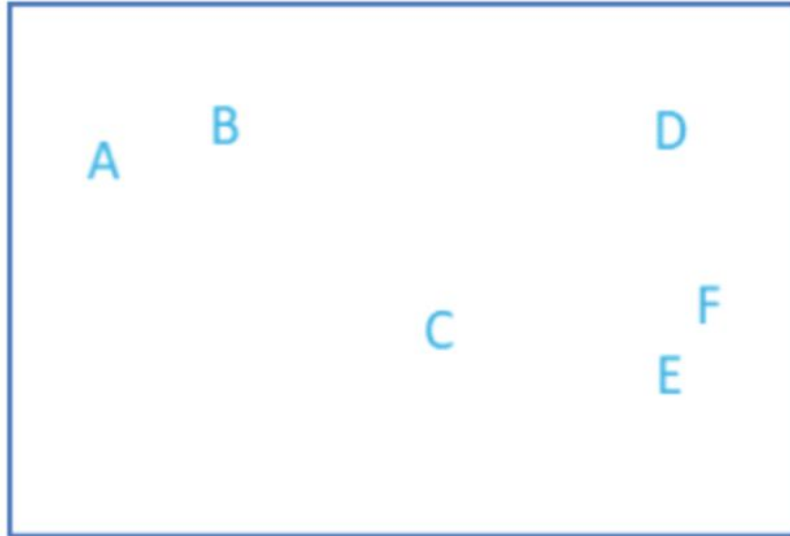


# Hierarchical Clustering Analysis (HCA)

## What is it?



# Hierarchical Clustering Analysis (HCA) Representation



Dendrogram



# Understanding Hierarchical Clustering

```
[ ] customer_data = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/Artificial Intelligence/research code/Mall_Customers.csv")
customer_data.head()
```

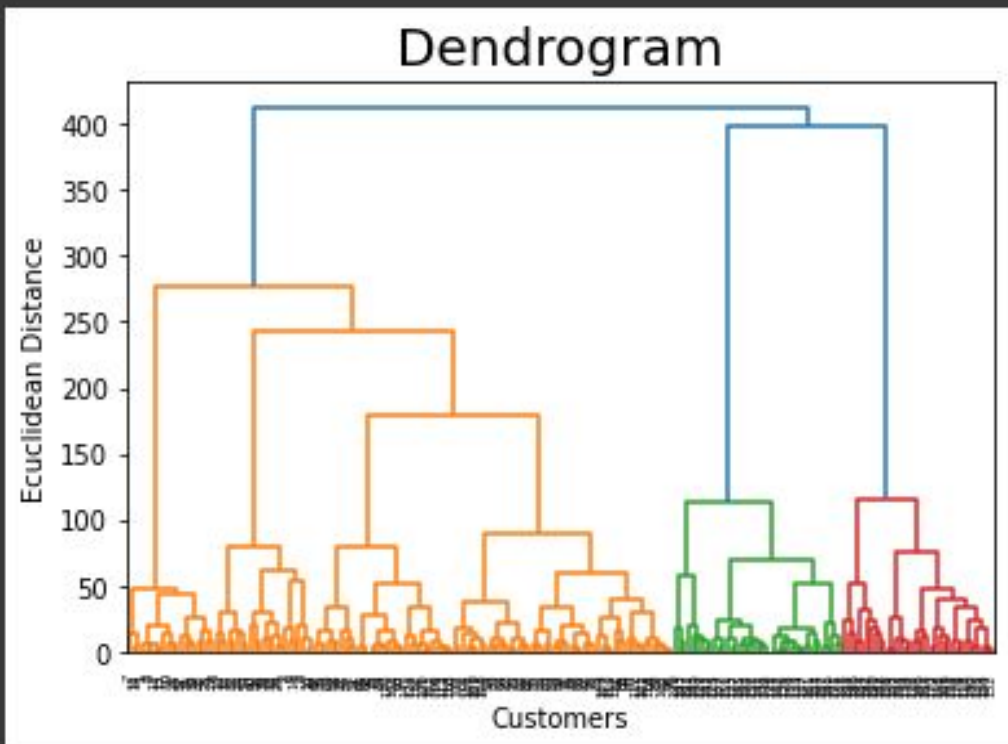
|   | CustomerID | Genre  | Age | Annual Income (k\$) | Spending Score (1-100) |
|---|------------|--------|-----|---------------------|------------------------|
| 0 | 1          | Male   | 19  | 15                  | 39                     |
| 1 | 2          | Male   | 21  | 15                  | 81                     |
| 2 | 3          | Female | 20  | 16                  | 6                      |
| 3 | 4          | Female | 23  | 16                  | 77                     |
| 4 | 5          | Female | 31  | 17                  | 40                     |

```
[ ] features = customer_data.iloc[:, 2:5]
features.head()
```

|   | Age | Annual Income (k\$) | Spending Score (1-100) |
|---|-----|---------------------|------------------------|
| 0 | 19  | 15                  | 39                     |
| 1 | 21  | 15                  | 81                     |
| 2 | 20  | 16                  | 6                      |
| 3 | 23  | 16                  | 77                     |
| 4 | 31  | 17                  | 40                     |

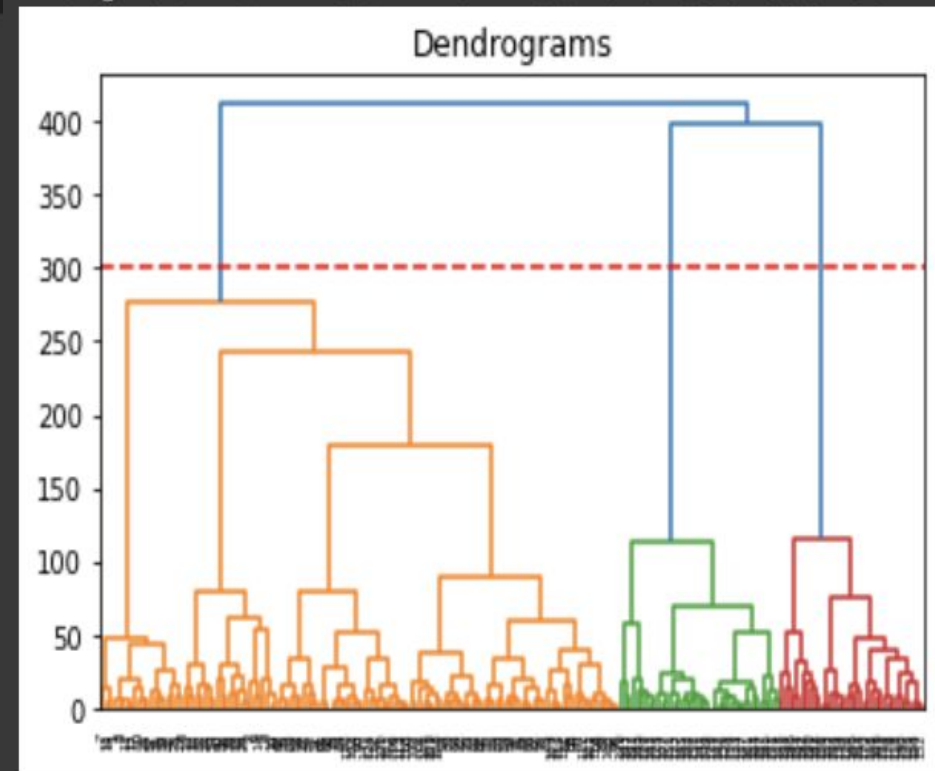
# Understanding Hierarchical Clustering

```
dendrogram = shc.dendrogram(shc.linkage(features, method='ward'))  
plt.title('Dendrogram', fontsize = 20)  
plt.xlabel('Customers')  
plt.ylabel('Euclidean Distance')  
plt.show()
```



```
plt.title("Dendrograms")  
dend = shc.dendrogram(shc.linkage(features, method='ward'))  
plt.axhline(y=300, color='r', linestyle='--')
```

<matplotlib.lines.Line2D at 0x7fe3c2d93250>

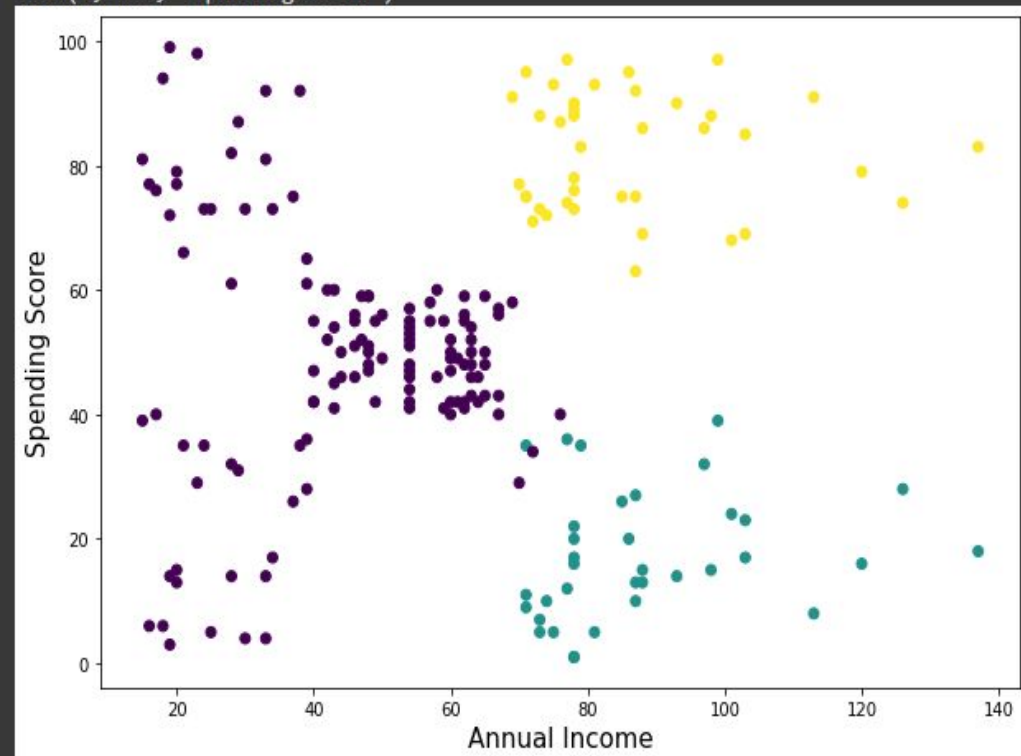


# Understanding Hierarchical Clustering

```
cluster = AgglomerativeClustering(n_clusters=3, affinity='euclidean', linkage='ward')
cluster.fit_predict(features)
```

```
plt.figure(figsize=(10, 7))
plt.scatter(features['Annual Income (k$)'], features['Spending Score (1-100)'], c=cluster.labels_)
plt.xlabel('Annual Income', fontsize=15)
plt.ylabel('Spending Score', fontsize=15)
```

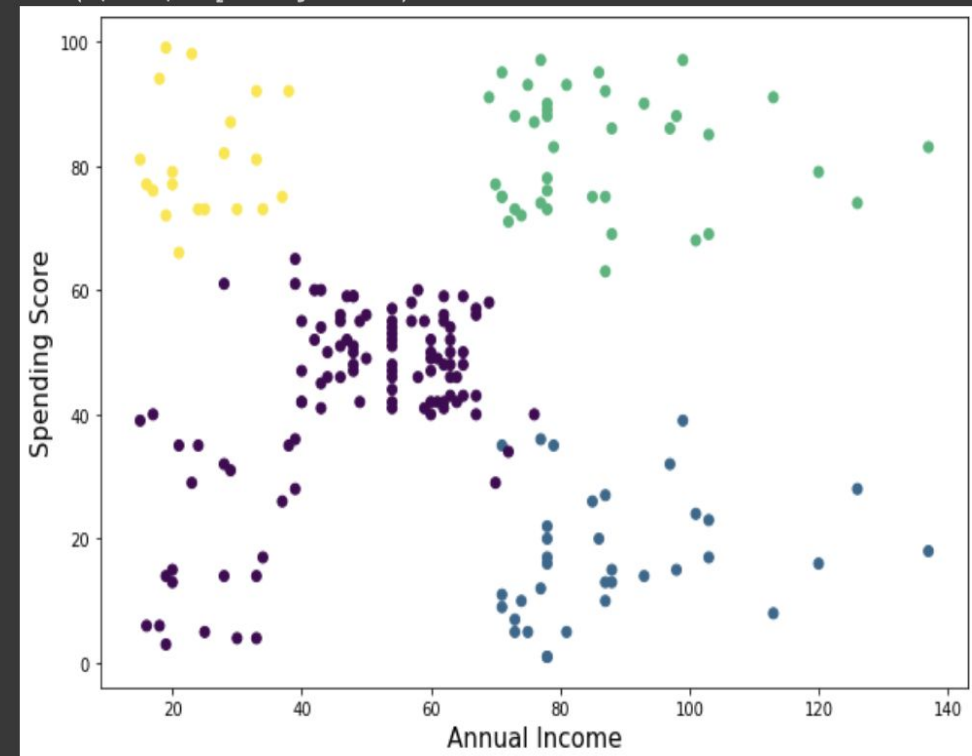
```
Text(0, 0.5, 'Spending Score')
```



```
cluster = AgglomerativeClustering(n_clusters=4, affinity='euclidean', linkage='ward')
cluster.fit_predict(features)
```

```
plt.figure(figsize=(10, 7))
plt.scatter(features['Annual Income (k$)'], features['Spending Score (1-100)'], c=cluster.labels_)
plt.xlabel('Annual Income', fontsize=15)
plt.ylabel('Spending Score', fontsize=15)
```

```
Text(0, 0.5, 'Spending Score')
```



# Consumer Behavior - Isolation Forest

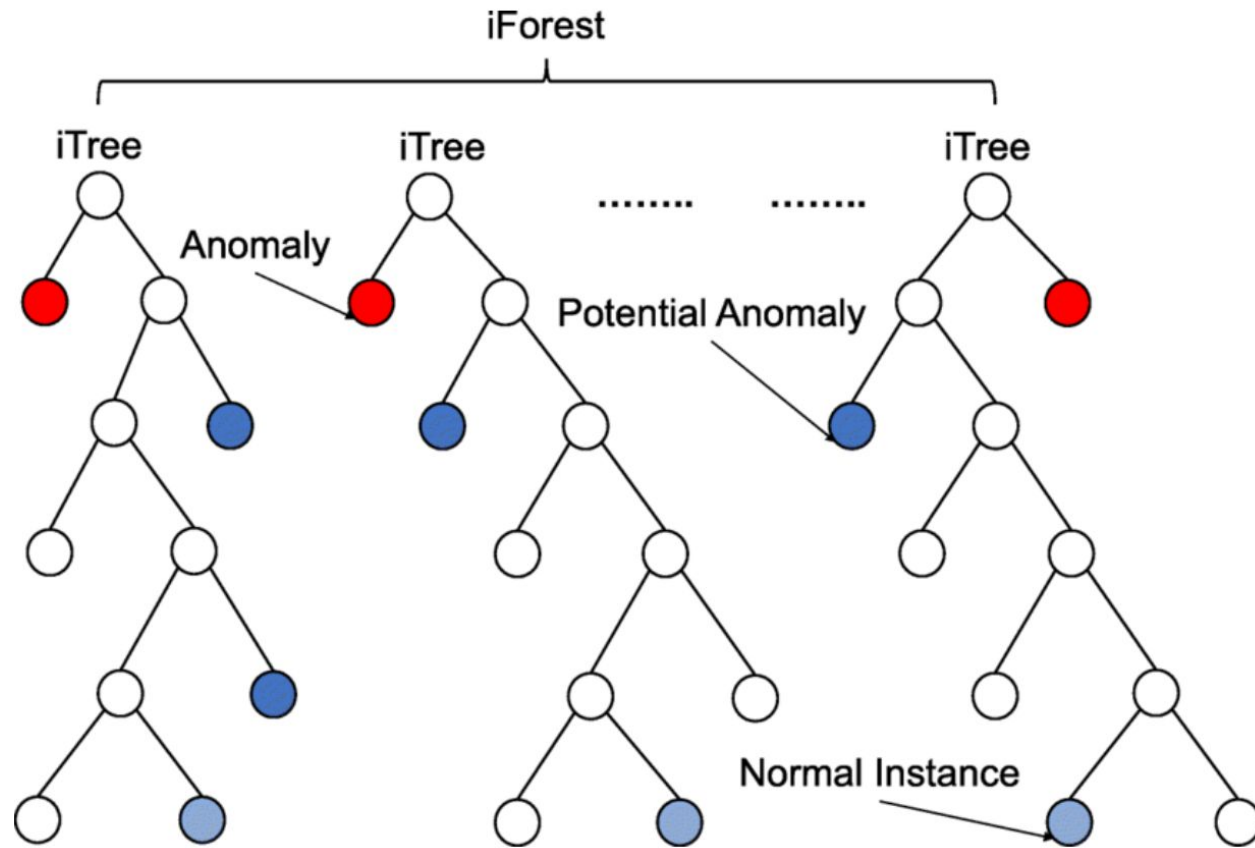
- ▶ Up until now our focus has been on clustering-based algorithms.
- ▶ The method I have researched is different in that it uses decision trees for anomaly detection.
- ▶ The idea was proposed by Fei Tony Liu, Kai Ming, and Zhi-hua in 2008 and has since been upgraded by several people to achieve even better results.

# Isolation Forest - Credit Card Fraud

- ▶ Good customer behavior can be used to point out bad customer behavior.
- ▶ A dataset of credit card transactions and their properties can be used to train a model to spot fraudulent transactions via isolation forest algorithm.
- ▶ The algorithm has plenty of other use cases.

# Isolation Forest - How it works

- 1.) Each iTree starts with a random sample of the original dataset.
- 2.) Nodes are partitioned by random variables and average thresholds.
- 3.) Splitting continues recursively until all data has been isolated or max depth has been reached.
- 4.) Forest of trees is used to assign anomaly score to new data.



# Dataset - Credit Card Fraud Detection (Kaggle)

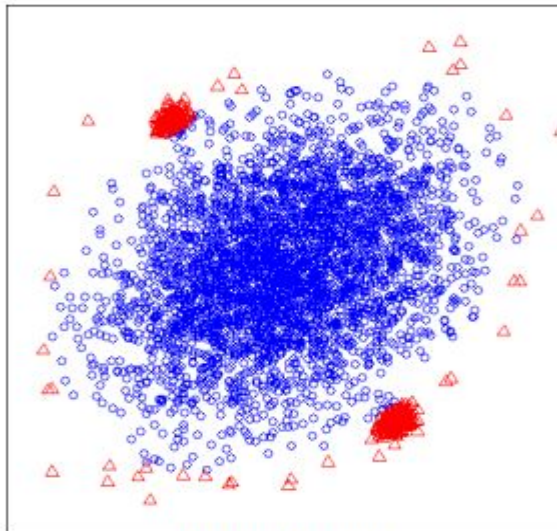
- ▶ 284,807 samples. 492 of which are classified as fraudulent.
- ▶ 30 variables. Details of 28 of them cannot be disclosed.
- ▶ Very imbalanced dataset but IF says THAT IS OKAY. why? Because IF does not need anomalous data in the training set to detect it in the testing data.

|   | Time | V1        | V2        | V3       | V4        | V5        | V6        | V7        | V8        | V9        | ... | V21       | V22       | V23       | V24       | V25       | V26       | V27       | V28       | Amount | Class |
|---|------|-----------|-----------|----------|-----------|-----------|-----------|-----------|-----------|-----------|-----|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|--------|-------|
| 0 | 0.0  | -1.359807 | -0.072781 | 2.536347 | 1.378155  | -0.338321 | 0.462388  | 0.239599  | 0.098698  | 0.363787  | ... | -0.018307 | 0.277838  | -0.110474 | 0.066928  | 0.128539  | -0.189115 | 0.133558  | -0.021053 | 149.62 | 0     |
| 1 | 0.0  | 1.191857  | 0.266151  | 0.166480 | 0.448154  | 0.060018  | -0.082361 | -0.078803 | 0.085102  | -0.255425 | ... | -0.225775 | -0.638672 | 0.101288  | -0.339846 | 0.167170  | 0.125895  | -0.008983 | 0.014724  | 2.69   | 0     |
| 2 | 1.0  | -1.358354 | -1.340163 | 1.773209 | 0.379780  | -0.503198 | 1.800499  | 0.791461  | 0.247676  | -1.514654 | ... | 0.247998  | 0.771679  | 0.909412  | -0.689281 | -0.327642 | -0.139097 | -0.055353 | -0.059752 | 378.66 | 0     |
| 3 | 1.0  | -0.966272 | -0.185226 | 1.792993 | -0.863291 | -0.010309 | 1.247203  | 0.237609  | 0.377436  | -1.387024 | ... | -0.108300 | 0.005274  | -0.190321 | -1.175575 | 0.647376  | -0.221929 | 0.062723  | 0.061458  | 123.50 | 0     |
| 4 | 2.0  | -1.158233 | 0.877737  | 1.548718 | 0.403034  | -0.407193 | 0.095921  | 0.592941  | -0.270533 | 0.817739  | ... | -0.009431 | 0.798278  | -0.137458 | 0.141267  | -0.206010 | 0.502292  | 0.219422  | 0.215153  | 69.99  | 0     |

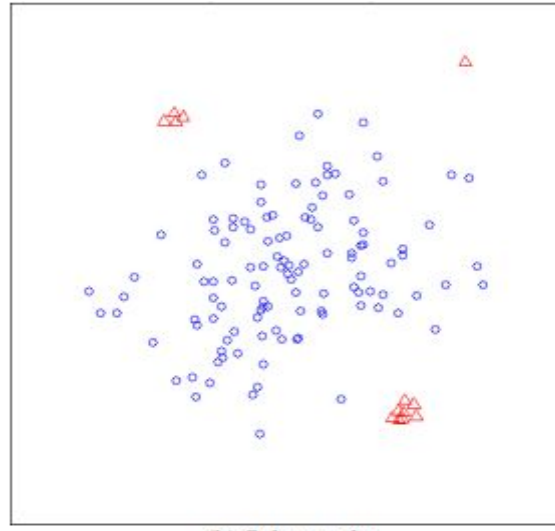
[5 rows x 31 columns]

# Isolation Forest - Strengths

- ▶ Scalable - builds models using a small sample size.
- ▶ Default configurations work on a lot of datasets.
- ▶ Handles masking and swamping problems nicely.
- ▶ Does not need to compute distance or density, improving computational requirements.



(a) Original sample  
(4096 instances)



(b) Sub-sample  
(128 instances)

# Isolation Forest - Weaknesses

- ▶ According to some literature, standard iForest has issues with...
- ▶ Handling “hard” anomalies - Uses a linear axis-parallel isolation method that only considers one dimension each time.
- ▶ “Ghost region” problem - inaccurate scores are assigned in certain regions.

# Isolation Forest on Steroids

- ▶ Extended Isolation Forest:
  - Uses random slope for each branching point.
  - Aims to fix the ghost region problem.
- ▶ Deep Isolation Forest:
  - Implements CERE for random representation.
  - Uses new anomaly scoring function, DEAS.

# REFERENCES

- ▶ [1]. Asniar and K. Surendro, "Predictive Analytics for Predicting Customer Behavior," *2019 International Conference of Artificial Intelligence and Information Technology (ICAIIIT)*, 2019, pp. 230-233, doi: 10.1109/ICAIIIT.2019.8834571.
- ▶ [2]. K.Tabianan, S. Velu, V. Ravi, "K-Means Clustering Approach for Intelligent Customer Segmentation Using Customer Purchase Behavior Data," *"Sustainability 2022, 14, 7243. <https://doi.org/10.3390/su14127243>*
- ▶ [3]. L. Jing, S. Pan, and L. Huang. "A Machine Learning Based Method for Customer Behavior Prediction," *Tehnički vjesnik* 26.6 ,2019,pp 1670-1676. <https://doi.org/10.17559/TV-20190603165825>
- ▶ [4]. E. Inzaugarat, "Using machine learning to understand customers behavior," *Towards Data Science*, 2019.
- ▶ [5]. A. S. M. S. Hossain, "Customer segmentation using centroid based and density based clustering algorithms," *2017 3rd International Conference on Electrical Information and Communication Technology (EICT)*, 2017, pp. 1-6, doi: 10.1109/EICT.2017.8275249.
- ▶ [6]. Severino F. Galán, "Comparative evaluation of region query strategies for DBSCAN clustering", *Information Sciences*, Volume 502, 2019, Pages 76-90, ISSN 0020-0255, <https://doi.org/10.1016/j.ins.2019.06.036>.
- ▶ [7]. S. -S. Li, "An Improved DBSCAN Algorithm Based on the Neighbor Similarity and Fast Nearest Neighbor Query," in *IEEE Access*, vol. 8, pp. 47468-47476, 2020, doi: 10.1109/ACCESS.2020.2972034.

# MORE REFERENCES

- ▶ [8]. F. Tony Liu & K. Ting & Z. Zhou. (2008). Isolation Forest. 413 - 422. 10.1109/ICDM.2008.17.
- ▶ [9]. Y. Chabchoub, M. U. Togbe, A. Boly and R. Chiky, "An In-Depth Study and Improvement of Isolation Forest," in IEEE Access, vol. 10, pp. 10219-10237, 2022, doi: 10.1109/ACCESS.2022.3144425.
- ▶ [10]. S. Ounacer, H. A. E. Bour, Y. Oubrahim, M. Y. Ghoumari, and M. Azzouazi, "Using isolation forest in anomaly detection: The case of credit card transactions," *Periodicals of Engineering and Natural Sciences (PEN)*, 02-Nov-2018. [Online]. Available: <http://pen.ius.edu.ba/index.php/pen/article/view/533/0>. [Accessed: 22-Oct-2022].
- ▶ [11]. S. Ounacer, H. A. E. Bour, Y. Oubrahim, M. Y. Ghoumari, and M. Azzouazi, "Using isolation forest in anomaly detection: The case of credit card transactions," *Periodicals of Engineering and Natural Sciences (PEN)*, 02-Nov-2018. [Online]. Available: <http://pen.ius.edu.ba/index.php/pen/article/view/533/0>. [Accessed: 22-Oct-2022].
- ▶ [12] A. S, "Customer segmentation using cluster analysis," Medium, 25-Nov-2020. [Online]. Available: <https://medium.com/analytics-vidhya/customer-segmentation-using-cluster-analysis-ed1f3a7c5920>. [Accessed: 22-Oct-2022].

# MORE REFERENCES

- ▶ [13] “Customer segmentation using hierarchical clustering,” enjoyalgorithms. [Online]. Available: <https://www.enjoyalgorithms.com/blog/customer-segmentation-using-hierarchical-clustering>. [Accessed: 22-Oct-2022].
- ▶ [14] C. K. H. Lee, Y. K. Tse, M. Zhang, and J. Ma, “Analysing online reviews to investigate customer behaviour in the sharing economy: The case of airbnb,” *Information Technology & People*, 07-Aug-2019. [Online]. Available: [https://www.emerald.com/insight/content/doi/10.1108/ITP-10-2018-0475/full/html?casa\\_token=XtAZ8l-UcTkAAAAA%3AOQmUnr1\\_QbNGL\\_LK408jqQ-vAv5NJQ5baF0UQnA0SBNcnHIRekP-2I9RccPqT83HAK2L812WB4OggdKqaSiZGQN7RsBUQdgs31gqI5xVSb0gWL0FAYDoDg](https://www.emerald.com/insight/content/doi/10.1108/ITP-10-2018-0475/full/html?casa_token=XtAZ8l-UcTkAAAAA%3AOQmUnr1_QbNGL_LK408jqQ-vAv5NJQ5baF0UQnA0SBNcnHIRekP-2I9RccPqT83HAK2L812WB4OggdKqaSiZGQN7RsBUQdgs31gqI5xVSb0gWL0FAYDoDg). [Accessed: 22-Oct-2022].
- ▶ [15] Mihir, “Locally linear embedding (LLE): Data Mining,” *Medium*, 19-Oct-2019. [Online]. Available: <https://medium.com/analytics-vidhya/locally-linear-embedding-lle-data-mining-b956616d24e9>. [Accessed: 22-Oct-2022].
- ▶ [16] Benediktdietz, “LocallyLinearEmbedding/\_config.yml at master · Benediktdietz/locallylinearembd,” *GitHub*. [Online]. Available: [https://github.com/benediktdietz/LocallyLinearEmbedding/blob/master/\\_config.yml](https://github.com/benediktdietz/LocallyLinearEmbedding/blob/master/_config.yml). [Accessed: 22-Oct-2022].